

SGBA: A stealthy scapegoat backdoor attack against deep neural networks

Ying He, Zhili Shen, Chang Xia, Jingyu Hua^{*}, Wei Tong^{*}, Sheng Zhong

Dept. Computer and Science Technology, Nanjing University, Nanjing, China

ARTICLE INFO

Keywords:

Backdoor attack
Deep neural network
Scapegoat
Data poisoning
Weight limitation

ABSTRACT

Outsourced deep neural networks have been demonstrated to suffer from patch-based trojan attacks, in which an adversary poisons the training sets to inject a backdoor in the obtained model so that regular inputs can be still labeled correctly while those carrying a specific trigger are falsely given a target label. Due to the severity of such attacks, many backdoor detection and containment systems have recently been proposed for deep neural networks. One major category among them are various model inspection schemes, which hope to detect backdoors before deploying models from non-trusted third-parties. In this paper, we show that such state-of-the-art schemes can be defeated by a so-called Scapegoat Backdoor Attack, which introduces a benign scapegoat trigger in data poisoning to prevent the defender from reversing the real abnormal trigger. In addition, it confines the values of network parameters within the same variances of those from clean model during training, which further significantly enhances the difficulty of the defender to learn the differences between legal and illegal models through machine-learning approaches. Our experiments on 3 popular datasets show that it can escape detection by all five state-of-the-art model inspection schemes. Moreover, this attack brings almost no side-effects on the attack effectiveness and guarantees the universal feature of the trigger compared with original patch-based trojan attacks.

1. Introduction

Deep Neural Networks (DNN) show a strong learning ability due to their deep network architectures. As a result, they have been widely applied in many tasks such as image classification and recognition (Ciregan et al., 2012), speech recognition and machine translation (Gibadullin et al., 2021) as well as text classification (Gargiulo et al., 2019). DNN usually needs a large number of samples to train the model. Be it for collecting or labeling data, large resources are consumed. Besides, training a model also requires high computing abilities of both CPUs and GPUs. Therefore, many users choose to outsource the training process to third-parties or download off-the-shelf models.

As not all the third-parties are trustworthy, such models are demonstrated to suffer from *Patch-based Trojan Attacks* (PTAs) (Chen et al., 2017; Gu et al., 2019; Liu et al., 2018), in which a malicious model trainer may inject a backdoor in the model so that regular inputs are still labeled correctly while those carrying a specific trigger will always be given a false target label regardless of their true ones. Here, a trigger is a small but arbitrary pattern (e.g., a small square) specified by the at-

tacker. The backdoors could be planted by poisoning the training data with a partially mislabeled data stamped with the trigger.

As trojan models may raise serious security threats, they have received extensive attentions and many containment proposals have been discussed in the literature (Chen et al., 2019a,b; Chou et al., 2020; Doan et al., 2020; Gao et al., 2019; Guo et al., 2019; Huang et al., 2020; Kolouri et al., 2020; K. Liu et al., 2018; Y. Liu et al., 2019; Tran et al., 2018; Wang et al., 2019, 2020; Xu et al., 2021). The most promising strategy is to deploy a model inspection scheme that aims to detect the anomalies due to the existence of backdoors before really applying the model in practice. According to existing works, the SOTA DNN model inspection schemes against backdoors can be briefly divided into two categories (Goldblum et al., 2021). The first category (Guo et al., 2019; Y. Liu et al., 2019; Wang et al., 2019, 2020) attempts to reverse engineer a trigger pattern for each class that can cause the misclassifications of all the inputs carrying it to the corresponding class, through various optimization-based methods. As the backdoor actually creates a ‘shortcut’ to the target class, they assume the reversed trigger of the target would be abnormal compared with those of normal classes and thus be detected. However, simple search strategies in the reverse engi-

^{*} Corresponding authors.

E-mail addresses: guapi7878@gmail.com (Y. He), zhilishen@163.com (Z. Shen), changxia656569@gmail.com (C. Xia), huajingyu@nju.edu.cn (J. Hua), wtong@nju.edu.cn (W. Tong), zhongsheng@nju.edu.cn (S. Zhong).

<https://doi.org/10.1016/j.cose.2023.103523>

Received 4 July 2023; Received in revised form 8 September 2023; Accepted 1 October 2023

Available online 5 October 2023

0167-4048/© 2023 Elsevier Ltd. All rights reserved.

neering can be stuck in sub-optimums. Although some of them enhance the search object restriction, we will prove it still can be misled to sub-optimums in the paper. The second category (Huang et al., 2020; Kolouri et al., 2020; Xu et al., 2021) employs the machine learning technique to recognize trojaned models. In particular, they first build and label both a large number of clean and trojaned models, and then take them as a training set to learn a binary classifier that can automatically differentiate benign and trojaned models with a high confidence. These defense schemes rely on the differences learned from benign models and backdoored models of the training set. We will show that once the malicious model is close enough to benign models, the learning ability of the detection classifier would reduce.

In this paper, we propose a stealthy *Scapegoat Backdoor Attack* (SGBA), utilizing patch triggers. We choose PTA owing to its robust extensibility in the physical world. Our research and observations indicate that SGBA can successfully evade detection by all current model inspection schemes. Note that, recently, some researchers have proposed some novel DNN trojan attacks that can really fool these schemes (Cheng et al., 2021; Gibadullin et al., 2021; Lin et al., 2020; Liu et al., 2020; Nguyen and Tran, 2021; Quiring and Rieck, 2020; Rakin et al., 2020; Tang et al., 2021; Wang et al., 2022a; Xu et al., 2023; Xu, 2020). Unfortunately, most of them more or less change the original definition of PTAs. For instance, in the composition attack (Lin et al., 2020), the trigger can no longer be an arbitrary pattern, but should be benign subjects. Moreover, in both this attack and the TaCT attack (Tang et al., 2021), the universal of the effective domain of a single trigger decreases. In the original PTAs, a trigger is universal and can affect all the inputs carrying it. We think the loss of such a universal feature brings a non-negligible sacrifice to the attack feasibility. In addition, some others are difficult to deploy in reality. For example, BATT proposed by Xu et al. (2023) uses a specific rotation angle as a trigger. However, it is hard to rotate real traffic signs on the road.

In particular, to defeat the trigger reverse-engineering schemes, our attack introduces a benign scapegoat trigger in data poisoning to hide the real trigger, which is constituted by two separated but interdependent patterns (both are small and arbitrary shapes) specified by the adversary. It places a loose restriction on the relative positions of the scapegoat and the real trigger on the input pixel space, demonstrating that it can misguide the existing trigger reverse-engineering methods to just reverse the scapegoat. In our proposal, a scapegoat is actually a benign trigger reversed from a clean model and thus will not raise any alarms. Moreover, during the training of the malicious model, we further confine the values of network parameters within the same variances as those of a clean model. According to our analysis, the weight variances of malicious models are much bigger than those of benign models especially in the latter layers, which might be because the malicious model has to make some weights unusually higher in order to react to the trigger. Our restriction can thus significantly reduce the anomaly of the obtained model compared with the clean ones, which obviously largely enhances the learning difficulty of ML-based detection mechanisms to differentiate them. In addition, our experiments prove that the proposed attack can still defeat such ML-based inspection schemes even when they know our attack and include our malicious models in the training set.

In summary, we make the following contributions:

- We propose a novel stealthy patch-based trojan attack against DNN in the training outsourcing scenario. It does not change the original concept of PTAs and can guarantee the universal effective domain of the trigger. Moreover, it introduces a scapegoat trigger to hide the real one and restricts the deviation of the model parameters from a clean model during training.
- We analyze and demonstrate the effectiveness of the proposed attack against all the existing model inspection mechanisms including those based on trigger reverse-engineering and those based on machine learning.

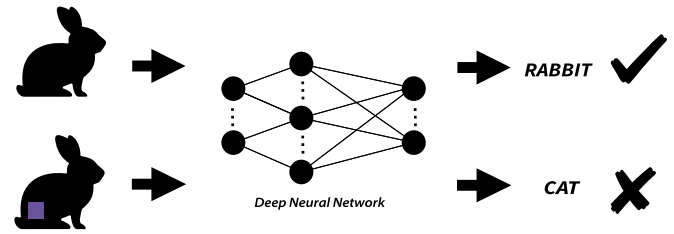


Fig. 1. A simple example of backdoor attacks.

- We apply our attack to three popular datasets of image classification including MNIST (LeCun et al., 2018), CIFAR10 (Krizhevsky and Hinton, 2009) and GTSRB (Stallkamp et al., 2012). The experimental results demonstrate that the side-effect of SGBA on classification accuracy is negligible. The average detection rates of SGBA significantly decrease from more than 90% to less than 10% compared with BadNets (Gu et al., 2019) (e.g. the most basic PTA) while SGBA keeps the original aggressivity at the same time. Besides, we prove that our attack can be extrapolated to bigger datasets like ImageNet (Deng et al., 2009).

2. Preliminary

Before explaining our attack scheme, it is necessary to first introduce basic backdoor attacks and mainstream model inspection schemes. Additionally, We conduct research on other related works and analysis why they are not taken into consideration.

2.1. Basic backdoor attacks

As artificial flaws hidden in the system, backdoors can leak information to the attacker or help the latter obtain access that does not belong to him or her. Backdoor attacks in DNN means there is a backdoor placed in the target model by an attacker. It is invisible when the model works on a normal dataset while can be triggered by the inputs with a special pattern, which is designed by the attacker, attached. Fig. 1 demonstrates a simple example of backdoor attacks. The model identifies the coming input as ‘rabbit’ in a normal situation which is correct and conforms to the design rule. When the same input comes with a square sticker, the model identifies it as ‘cat’ which is absolutely wrong. It is different from common-cause recognition failures of model learning capacity which tend to appear randomly. It’s an artificial misleading recognition where inputs with the sticker tend to be identified as the same result that is usually designed by the attacker.

The basic manipulation of backdoor attacks is to adjust the weights of the target model by data poisoning with a special trigger injected into the training set. The trigger is designed by the attacker and often has its fixed properties like shape and location in the field of image classification. This basic type of backdoor attacks is what we call PTAs. The workflow of PTAs is demonstrated in Fig. 2. BadNets proposed by Gu et al. (2019) is the most basic PTA. What needs to be noted is that the backdoor attack can be applied into many other fields like speech recognition besides image classification. But image classification is still the most widely discussed topic in the field of backdoor attacks and also what we are only concerned with in this paper.

Some popular and ingenious extension of backdoor attacks using data poisoning have been proposed in the past few years. Gu et al. (2019) propose a single-target backdoor attack and an all-to-all attack in 2017 where the single-target attack means that there is only one backdoor in the target model that misleads the result into one fixed label and the all-to-all attack means that every class of the target model is correspondingly misled to another class one-to-one. Chen et al. (2017) mention a blended injection attack which uses a background image as the trigger to poison the training set by mixing the clean data with the trigger according to a certain proportion assigned by the attacker

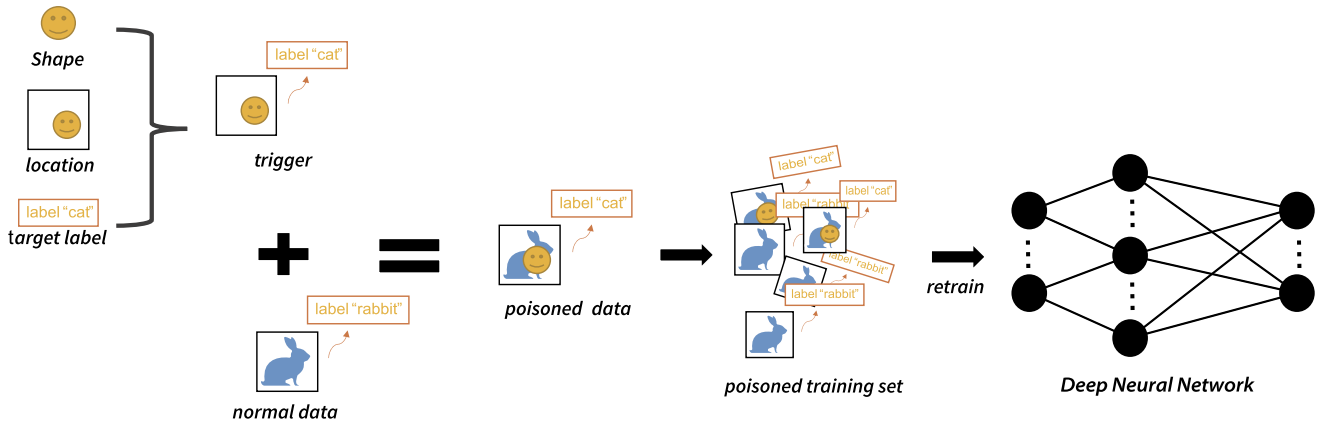


Fig. 2. Poisoning and training workflow of PTAs.

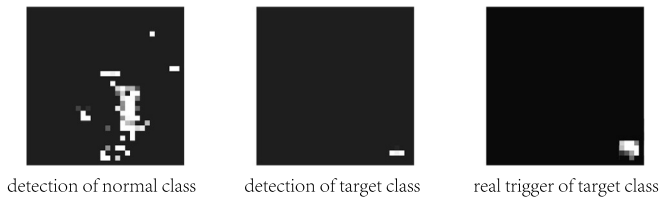


Fig. 3. A simple example of Neural Cleanse's reverse results.

instead of replacing part of the clean data with the trigger. Another way to deploy backdoor attacks is to adjust the weights of the model directly. Liu et al. (2018) propose an attack that selects a set of neurons that can be extremely activated by the trigger and strengthens the connection between the neurons and the trigger. These attacks work well when they were first proposed and have now been limited since defense schemes like trigger reverse-engineering schemes and ML-based schemes have been put forward. From the perspective of attackers, more complicated and strong attacks need to be designed.

2.2. Mainstream model inspection schemes

Under the concerned scenarios, the existing model inspection schemes can be divided into two types: The trigger reverse-engineering schemes and the ML-based schemes (Goldblum et al., 2021). The first type of schemes usually generates reversed triggers for every class and compares their features to find out if there is an anomaly. The second type of schemes makes use of machine learning technologies to create a black-box classifier to learn the differences between benign and trojan models. The features extracted by the trigger reverse-engineering schemes are always concrete (e.g., the size of the reversed pattern) while the differences learned by the ML-based schemes are unexplainable. The SOTA detection methods for each type are demonstrated as follows.

Neural Cleanse (Wang et al., 2019) is the first and most representative of the trigger reverse-engineering schemes that detects the abnormal class from normal classes. It computes the optimal reversed triggers to convert all inputs to each target label and then checks if any trigger's $L1$ norm is significantly smaller than the others as the backdoor indicator. This is due to the fact that they thought a 'shortcut' was created by the backdoor from normal classes to the target class. The 'shortcut' can be reflected by the sizes of the reversed triggers (i.e., $L1$ norms). This kind of difference between the target class and normal classes in an infected model is what Neural Cleanse uses to detect whether backdoors exist or not. Fig. 3 is a simple example of the reverse result of Neural Cleanse. It shows that the reversed trigger of a normal class is much bigger than that of the target class. At the same time, the

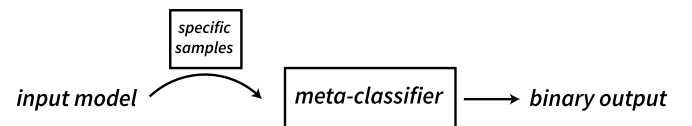


Fig. 4. Simple workflow of Meta Neural Trojaned Model Detection.

reversed trigger of the target class is similar to the real trigger used by the attacker.

TABOR (Guo et al., 2019) improves the regularization terms of the reverse-engineering goal on the basis of Neural Cleanse to narrow down the margin of search error. They think the reverse results of Neural Cleanse are unstable because they can easily fall into the sub-optimal solutions. TABOR takes more strict limitation to the size, smoothness, location and redundancy of the reversed trigger to exclude some unexpected items which are too big, too scattered, overlaying or blocking key features of the original images. For each target it designs a corresponding regularization term to the objective function. Such improvements help to filter some sub-optimums during searching process.

ABS (Y. Liu et al., 2019) has a different optimization method from Neural Cleanse and TABOR, adopting neurons stimulation analysis to optimize the reverse engineering. It stimulates neurons from different extents and adds the one into candidate collections once it generates an extremely high activation with some stimuli. ABS scans random layers of neural networks and identifies all candidate anomaly neurons first. Then it reverses triggers for every candidate neuron. If there is a reversed trigger that can make the corresponding candidate neuron always highly activated on the clean training set, the target model is classified as a trojaned model. Compared with Neural Cleanse and TABOR, ABS only needs a small clean dataset and costs less time.

Meta Neural Trojaned Model Detection (MNTD) (Xu et al., 2021) is the main outstanding of the ML-based schemes that are detecting benign and malicious models with machine learning technologies. It generates a large number of benign and trojaned models called as 'shadow models' to train a binary classifier. Since the strategy of taking the whole networks of shadow models as the training set does not possess a good performance, it uses special input samples called the query set to extract feature vectors from shadow models and use them to train the classifier model. The binary result of the classifier indicates whether the target model is backdoored or not. Fig. 4 shows the simple workflow of MNTD. Besides, to resist the adaptive attacks, MNTD proposes a robust scheme which uses random parameters for the binary classifier and only trains the query set.

These defense schemes limit attackers from specific and abstract aspects. Attackers now need to face more strict challenges.

2.3. Related works

Attack: There are other backdoor attacks on DNN related to us. Lin et al. (2020) construct the trigger with benign features of different classes. They can escape from Neural Cleanse and ABS. Their composite trigger is usually big and does not meet the detection key idea of Neural Cleanse whereby the trigger is considered to be small enough. Besides, benign features won't make neurons highly activated, thus they can escape from the detection of ABS. TaCT proposed in Tang et al. (2021) designs a more complex 'misclassification rule' that it requires its trigger is only triggered by a specific class. Through this way, TaCT reduces its model anomalies. B^3 (Gong et al., 2023) aims to conduct backdoor attacks on black-box machine learning models. It starts by creating a substitute model using public datasets and adversarial samples. During the design of the trigger generation algorithm, it identifies the neuron in the model most closely related to the target class and generates a backdoor trigger with the goal of activating that specific neuron at a high level. Liu et al. (2020) carefully design the trigger as a mirror object after reflection to achieve the stealthiness of it and keep the classification accuracy on the clean dataset. However, not all the reflection images can be backdoor triggers. A too small reflection object is not aggressive enough while a too strong reflection object breaks the stealthiness of the backdoor. Rakin et al. (2020) propose a new attack named TBT that does not need to intervene the training process of DNN. They inject backdoors by flipping some specific bits of weight parameters stored in DRAM. Their triggers are calculated to trigger corresponding bits. Besides, they need the access to the architecture of the computer. FooBaR (Breier et al., 2023) is similar to TBT. It also employs a hardware-based physical attack approach to inject backdoors. Its objective is to perform fault attacks on the activation functions of the target model during the training phase, thereby establishing a correlation between faults and the target class. During the inference phase, a small number of model parameters are used to compute specific noise for any input sample, causing the specific fault in the activation function, resulting in high-confidence classification into the target class. This attack requires the attacker to have physical access to the device running the model. Quiring and Rieck (2020) combine data poisoning and image-scaling to deploy new backdoor attack. There is no trigger in the vision until the poisoned image is downsampled. WaNet (Nguyen and Tran, 2021) uses a special warping as a trigger. It calculates a satisfied warping method to deal with all the target images. These warped images would be identified as another class. DFST (Cheng et al., 2021) transfers the style of the target images and uses the specific style as a trigger. It trains a trigger generator to add the specific style for images. At the same time, it deploys detoxification to its model. Additionally, Wang et al. (2022a) generate a dispersed trigger to disturb the pixels of images with low complexity, while BATT (Xu et al., 2023) proposes a stealthy backdoor that is only triggered when the image rotates to a specific angle.

Most of these attacks require a delicate trigger with too much limitation while SGBA allows the shape and pattern of the trigger to be free enough. Both of Lin et al. (2020) and Tang et al. (2021), the universal of their triggers decreases. TaCT (Tang et al., 2021) can only affect one specific class. Composite attack (Lin et al., 2020) is suit for the scenarios where the two benign objects of the trigger appear at the same time. When the attacker attacks the third class, she/he needs to attach both of the two benign objects on the images which would decrease the stealthiness and the attack success rate of the trigger. Compared with them, SGBA keeps good universal of the trigger. The trigger generated by B^3 (Gong et al., 2023) induces high activation in a specific neuron, which aligns with the compromised neurons detected by ABS. In contrast, SGBA avoids this issue and successfully eludes the detection of ABS. TBT attack (Rakin et al., 2020) and FooBaR (Breier et al., 2023) relies on hardware or physical access to devices while SGBA does not. As to Quiring and Rieck (2020), it aims to make backdoor attacks escape from eyes of human while our target is to escape from machines.

Moreover, these attacks do not take ML-based schemes into consideration.

Besides, schemes like Cheng et al. (2021); Nguyen and Tran (2021); Wang et al. (2022a); Xu et al. (2023) are hard to deploy in the reality. While a warped image can escape from human and machine, physically warping a real object is difficult. Similarly, it is challenging for DFST to modify the style of the real environments. Additionally, making subtle interference with different color channels of a real object or rotating a traffic sign on the road to a specific angle is also problematic. In comparison, the patch-based trigger of SGBA is more easily attached to the target object in real-world scenarios. Although DFST (Cheng et al., 2021) considers ULP (Kolouri et al., 2020), it avoids exposure itself to the training set of ULP (Kolouri et al., 2020), making it more prone to evading detection.

Defense: Besides the mainstream model inspection schemes studied in the paper, there are some other defense schemes. Practical Detection proposed by Wang et al. (2020) is also an improvement based on Neural Cleanse. Besides benign triggers for each class, it generates reversed triggers for each input. It claims a model is backdoored only when the reversed trigger of class A appears similar to the reversed trigger of each single input. Although the style of generating reversed triggers is changed, the search method does not. So this scheme would still find the scapegoat instead of the real trigger when detecting SGBA models and we haven't discussed it above. FeatureRE proposed by Wang et al. (2022b) is a reverse engineering detection aimed at feature-space triggers. However, it relies on the activation of a small number of compromised neurons, similar to the principle employed by ABS. Yet, our experiments demonstrate that SGBA does not induce distinct high-activation behavior in individual neurons. Consequently, this detection method is not applicable to our attack scheme. AEVA (Guo et al., 2022) is a backdoor detection algorithm designed for black-box models. Its key idea revolves around the discovery of an adversarial singularity phenomenon, where the values of adversarial perturbations generated for the infected labels exhibit a peaked distribution in pixel space, with their peak height surpassing that of perturbations generated for uninfected labels. AEVA leverages this phenomenon to identify backdoor models. However, its fundamental optimization algorithm for generating adversarial perturbations remains consistent with NC, hence AEVA still cannot circumvent the local optima caused by scapegoat triggers.

We do not take undifferentiated defense schemes like Fine-Pruning (K. Liu et al., 2018) into consideration because they directly remove the part that could be trojaned but cannot tell whether a model is malicious or not. Besides, they always cause a loss in model accuracy. Spectral Signatures (Tran et al., 2018) and Activation Clustering (Chen et al., 2019a) also do detection before model deploying. But they claim that the defense has rights to access training set and network weights of the target model. However, we suppose that the malicious third party only releases the target model because there is no benefit for them to release their poisoned training set, so we do not consider them in this paper. What's more, we don't take dynamic detections that detect backdoors during the running process into consideration either. SentiNet (Chou et al., 2020) and STRIP (Gao et al., 2019) both think that the trigger in the input contributes the most to the classification result. Februs (Doan et al., 2020) protects models by clearing the trigger in the input. Beatrix (Ma et al., 2023) employs Gramian information to measure and capture anomalies in poisoned samples. All of them need to check and detect every input while the model is running. We think that it will consume too much resources.

As to ABL (Li et al., 2021a) and DBD (Huang et al., 2022), they primarily focus on training a clean model using poisoned datasets. They suit situations where the owner trains the model using third-party datasets. However, they are not suitable for directly using off-the-shelf models obtained from third parties. CRAB (Ji et al., 2022) employs a technique of randomly splitting the image into pieces and calculating the average prediction results of each piece to determine the final result. However, this defensive approach comes at the cost of primary

task accuracy. The classification accuracy is reduced by approximately 10%. There are some other special defense schemes. Some works choose to design specialized hardware to assist detection (Javaheripi et al., 2020a,b). Kaviani et al. (2020) design a more complex constructure of DNN to protect models from backdoor attacks. Although these works are effective but they need to change the current way of DNN designation and application.

Other Areas: This paper focuses on traditional computer vision area in centralized DNN. Backdoor attacks are also widely discussed in many other areas like video recognition (Hammoud et al., 2023; Zhao et al., 2020), multi-label learning (Chen et al., 2023), and time series analysis (Jiang et al., 2023). Besides, wireless communication (Davaslioglu and Sagduyu, 2019), federated learning system (Abad et al., 2023; Zhang et al., 2019, 2021, 2023; Zhao et al., 2021) and Internet of Things (Qiu et al., 2021; Zhang et al., 2021) are concerned about backdoor attacks. We choose traditional computer vision because this field is still the most discussed. The researches on attack and defense of this field are more comprehensive, challenging and universal.

3. Threat model

3.1. Adversarial capabilities

In this paper, we assume that the attacker has the full control of the training workflow of the target DNN. She/he can also access and modify the training set. We think such an assumption is reasonable when the user outsources the training to the attacker or directly downloads an off-the-shelf model from a third-party market.

3.2. Defender's capabilities

We suppose that the defender has a clean dataset (below 10% of the clean training set) that can be used to test the performance of the model. She/he has a full access to all the parameters and weights of the target model, but has no way to interfere with the training process performed by the model provider, i.e., the attacker in this paper.

3.3. Adversarial goal

The objective of the adversary is to produce a model $F(\cdot)$ containing a stealthy patch-based backdoor by poisoning the training set. Note that although there are some other kinds of backdoors (Cheng et al., 2021; Li et al., 2021b; Nguyen and Tran, 2021) like human invisible noise (Li et al., 2021b) that can play an important role in some aspects, we still choose patch-based backdoor because it is easier to be extended to physical attacks. Here, a so-called stealthy patch-based backdoor satisfies the following requirements:

(1) It should act like a general patch-backdoor in BadNets (Gu et al., 2019): for any input $x \in \mathcal{X}$, once stamped with a trigger pattern τ , they will be misclassified to the target class C_t chosen by the attacker, i.e., $F(x + \tau) = C_t$. Otherwise, they will be correctly labeled. i.e., $F(x) = C_x$, where C_x is the true label of x . In other words, we should guarantee the universal feature of the trigger. In addition, it should be a small but arbitrary pattern (e.g., a small square) specified by the attacker.

(2) It can escape from being captured by the existing model inspection mechanisms, which include two types: the first one are trigger reverse-engineering schemes (e.g., Neural Cleanse and its improvements), and the second one are ML-based schemes (e.g., MNTD).

4. The proposed attack

We propose a new stealthy scapegoat backdoor attack which can evade the mainstream model inspection schemes. As mentioned earlier, these schemes can be briefly divided into two types: the trigger reverse-engineering schemes and the ML-based schemes (Goldblum et

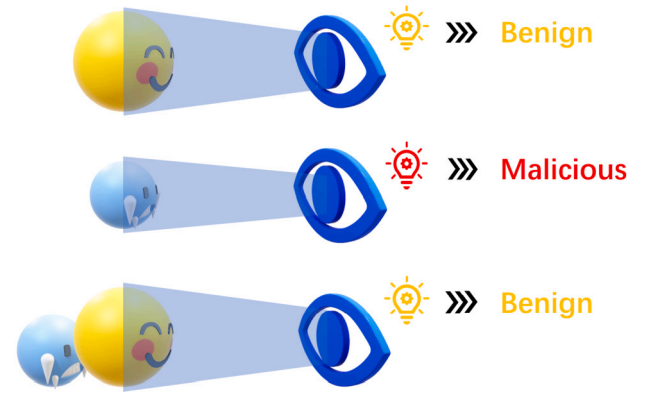


Fig. 5. Basic idea of the scapegoat attack.

al., 2021). In this section, we will first describe our basic methods to bypass them, respectively, and then integrate them to compose a complete robust and stealthy backdoor attack scheme.

The trigger reverse-engineering schemes are those that attempt to reverse engineer the triggers to mis-classify all the inputs to each inspected class and then determine the malicious nature of the model by comparing the reversed triggers. Note that for a clean model these schemes can still reverse a trigger for each class because in the extreme case the adversary could take an image directly from the target class as a trigger as long as it can fully cover the original inputs. We call such triggers of clean models *benign triggers*, the sizes of which are considered to be close in the same model. For a malicious model with a backdoor, however, these schemes consider that there should be an abnormal reversed trigger that is much more compacted than others because the backdoor creates a ‘shortcut’ from other classes to the target. This abnormal one is actually a close approximation of the real trigger created for the backdoor. The above is the common idea of the schemes in this category and their major difference lies in the optimization methods for trigger reverse engineering. In the following paragraphs, we first propose our attack scheme and then analyze its effectiveness against different trigger reverse-engineering schemes.

4.1. How to bypass the trigger reverse-engineering schemes?

To evade the detection of the above trigger reverse-engineering schemes, the basic idea of our proposed backdoor attack is to introduce a so-called “scapegoat” trigger to hide the real trigger as Fig. 5 shows. More specifically, it aims to misguide the optimization methods of the trigger reverse-engineering to find a scapegoat trigger that is intentionally exposed rather than the real one. This scapegoat is dressed up to look perfectly benign and thus would not trigger any alarms. Obviously, the success of this attack depends on the following two conditions:

- It can successfully cheat the trigger reverse-engineering methods of existing defense schemes to reverse the scapegoat trigger for the target class instead of the real one.
- All the triggers (including the scapegoat one) reversed for each class should be considered normal and will not raise any alarms.

In order to meet the above two conditions, the proposed scapegoat attack is composed of two indispensable components as Fig. 6 shows:

(1) Use the benign triggers of a clean model to create the scapegoat: As the triggers reversed from a clean model must be benign, they are ideal choices for scapegoats. So, in this step, the attacker first trains a clean model and reverse-engineers the benign trigger of each class using the same method as Neural Cleanse (Wang et al., 2019), namely, by finding a pattern that is as small as possible but with high enough attack potential. Then, she/he chooses a part of the samples from the training set randomly and attaches benign triggers onto them separately. More-

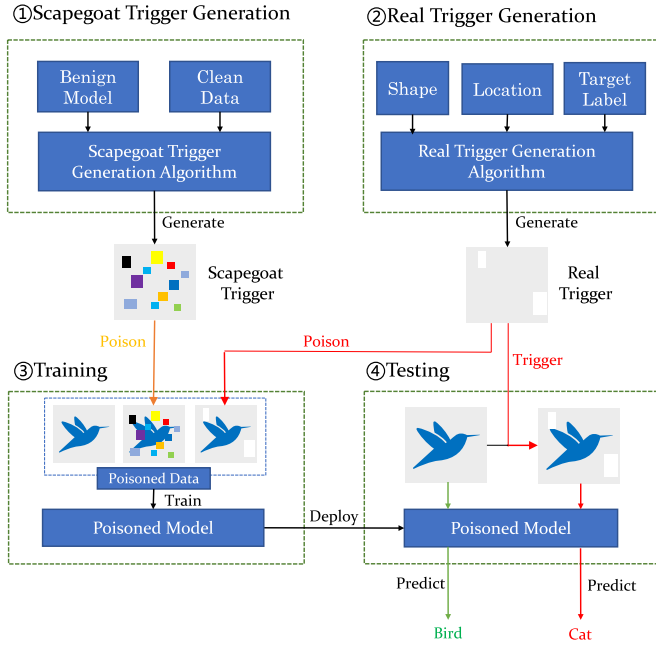


Fig. 6. The two key components of the scapegoat attack: the scapegoat and the real trigger.

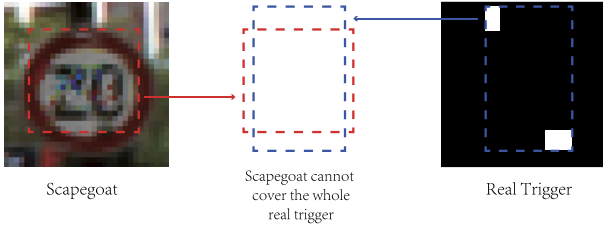


Fig. 7. The distribution area of the scapegoat cannot cover the two parts of the real trigger at the same time.

over, their original labels are changed to the corresponding classes of triggers. These samples together with those poisoned by the real trigger and the original ones constitute the final training set. In particular, the scapegoat trigger in our proposal is just the benign trigger of the target class. Additionally, the benign triggers of other classes here are used to ensure that the reversed trigger of the target class does not deviate significantly from those of other classes after poisoning.

(2) **Split the real trigger into two interdependent parts:** The real trigger is composed of two separated patterns, each of which can be freely specified by the attacker as traditional triggers. In particular, we require them to obey the following two conditions at the same time:

- The backdoor is triggered only when the two parts exist together. It requires that the inputs carrying only one part of the real trigger are mis-classified to any labels other than the target.
- Although the relative positions of the two parts can be freely specified by the attacker, she/he has to guarantee that they cannot be simultaneously covered by the distribution area of the scapegoat as we show in Fig. 7.

The attacker injects such a fission trigger into the training set and modify the label of the injected images to the attacker-chosen target class.

Our proposal finally uses the whole training set to build the final backdoor model. During the prediction stage, the attacker only needs to attach the real trigger onto the input to manipulate the predicted outcome. We below refer to specific detection schemes to explain why

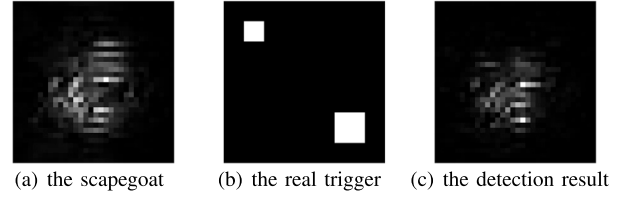


Fig. 8. A simple example of the optimization result.

this model can satisfy the two necessary conditions for escaping from being detected.

Effectiveness against original Neural Cleanse (NC) (Wang et al., 2019). In this scheme, the trigger reverse-engineering is regressed as an optimization process to search for the minimum pattern in the global pixel space that can mislabel all the inputs attached with it to a specific target class with a high confidence. In our attack, the scapegoat is actually a benign trigger pattern reversed with the same optimization process from the clean model. So, it can easily achieve the desired high misclassification rate after poisoning the training set. In addition, NC optimizes the size of the reversed trigger from large to small. Note that the size here refers to the total number of pixels that a trigger overwrites. In our attack, scapegoat of this size is much larger than the real one. Due to both of the above facts, there is an extremely high probability for the search algorithm to first catch the scapegoat. Worse still, as the real trigger is split into two parts that cannot be both covered by the scapegoat, it can only obtain a more precise pattern of the scapegoat no matter how it optimizes within the area of this sub-optimal result as Fig. 8 shows. Therefore, our attack satisfies the first necessary condition targeting NC.

We now consider the second condition. According to the above analysis, we know that the reversed trigger for the target class would be a close approximation of the scapegoat, which is just a benign trigger of a clean model and thus must look normal from itself. In addition, since we also inject benign triggers of other non-target classes into the training set for re-training, it guarantees that all the reversed triggers are close to those of the clean model, inter-comparisons of which should not show any anomaly either. Note that the existence of the real trigger may affect non-target classes and result in the raw triggers being reversed and changed to an anomaly. That's why we have to also inject benign triggers of these classes.

Effectiveness against various improvements of Neural Cleanse. Because NC may fall into a local optimum, defenders may improve its optimization algorithm to avoid this problem. According to our analysis, there are mainly two improvement strategies. The first one is to introduce some constraints on the optimization to avoid obtaining the local optimum. For instance, TABOR proposed by Guo et al. (2019), observes that the benign triggers (i.e., the sub-optimums) of a model are usually much more scattered and centrally located than the real trigger, usually a special and compacted pattern (e.g., a small smile face) placed by the attacker at an inconspicuous corner position. Therefore, it introduces 4 additional regularization terms according to such observations in the loss function of NC to avoid obtaining the unexpected sub-optimums. Unfortunately, to guarantee the attack effect of the reversed trigger, TABOR assigns very small initial weights to these terms, and gradually increase them only when the overall loss is optimized to a certain level. As a result, TABOR will be still lured to the sub-optimum (i.e., the scapegoat) with a high probability in the beginning, and then further optimizes within the area of scapegoat to find a more expected results with the additional regularization terms. Unfortunately, as in our design the distribution area of scapegoat cannot cover the both constituents of the real trigger, we believe TABOR can hardly reverse the real trigger either. Our experiments against TABOR in Sec. 5.3 will demonstrate this conclusion.

The second strategy clips the search area of the algorithm from the global pixel space of images to smaller ones that cannot cover the scape-

Table 1

A simple example of clipping approaches.

	BadNet	SGBA base	SGBA base (cutdown)	SGBA fission (cutdown)	SGBA fission (hollow)
Trigger Set					
Detection Result					
Detection Result of Normal Class					
Anomaly Index	4.34	0.39	1.39	0.68	0.01

goat, and thus escape from the local optimum. We consider two clipping approaches. The first one is to restrict the search scope to a rectangle that is sufficiently smaller than the size of the scapegoat and then slide this rectangle within the whole pixel space to find the global optimum. We call this approach *the cutting improvement*. Obviously, it does not work because our real trigger is split into two parts that cannot be covered by a rectangle smaller than the scapegoat at the same time. As a typical example shows in the 4th column in Table 1, when the search rectangle covers just one part of the real trigger, the results of the target class and the normal class are similar and the anomaly index of the target class is well below the threshold. In order to show that splitting the trigger into two parts is important for the scapegoat attack, we also give a normal trigger version in the 2nd and 3rd columns. From the 3rd column we can see, the normal trigger would be caught by the detection even we add the scapegoat to protect it once the search area is smaller than the scapegoat and happens to cover the area of the real normal trigger.

Inversely, the second clipping approach assumes the area being cut off is a rectangle and the search scope is restricted to the remaining space. We name this approach *the hollow improvement*. The location of the rectangle being cut could be tried at any position possible and its size should be smaller but not much smaller than that of scapegoat. In this case, the remaining space may happen to cover both constitutions of the real trigger while destroying the scapegoat. Unfortunately, the scapegoat is usually located near the center of the pixel space of images, which may overlap with the reversed benign patterns of normal classes with a significantly high probability. Therefore, such a cut would also destroy the spaces containing the normal benign trigger patterns of normal classes and what kinds of patterns could be reversed from the remaining space is hard to predict. According to our experiments, in this method, there will be no obvious anomaly in the reversed trigger of the target class compared with those of normal classes, as the example in 5th column of Table 1 shows. Besides, the anomaly index of the target class is even the smallest among all the classes in many cases.

Effectiveness against ABS (Y. Liu et al., 2019). ABS employs a totally different optimization process to reverse the triggers. It first intercepts internal neurons and enlarges their activation values to check if such stimulation can result in misclassification with high confidence. If such neurons are found, they are considered to be potentially exploited by the backdoor. The defender then tries to reverse corresponding triggers that can abnormally activate them, respectively. If one reversed trigger can cause poisoned inputs for other labels to a specific label consistently, this model is considered trojaned. Unfortunately, this scheme assumes only a single neuron will be maliciously triggered by the attack image, which is no longer true in our proposal: the two separated and distant parts of the trigger would activate more neurons with lower activation values.

In the experiment, we select four schemes, NC, clipping approaches, TABOR and ABS to prove the scapegoat attack is effective and stealthy. Detailed results are present in Sec. 5.

4.2. How to bypass the ML-based detection schemes?

ML-based detection schemes (Huang et al., 2020; Kolouri et al., 2020; Xu et al., 2021) aim to train a binary classifier (usually a deep model) to distinguish between benign and malicious models. For this purpose, they usually first build a large number of benign and trojan models as a training set. Then, a unique feature extraction method is applied to extract features from these positive and negative samples (i.e., the pre-trained benign and trojan models), which are finally used to train a precise binary classifier through existing deep learning techniques.

Researches have shown that directly taking the raw network parameters of a model as the input of the classifier cannot bring satisfying performance and accuracy. That is why an additional feature extraction method is still required even after the classifier is DNN-based. For instance, MNTD (Xu et al., 2021), which is a SOTA in this category, uses the outputs of the target model on a set of specially-optimized images as the final feature vector. In particular, these images are optimized to maximize the difference between their outputs on the benign and trojan models in the training set, and can be regarded as a kind of feature extractor in a sense, like Fig. 9 shows.

Obviously, the feasibility of such a DNN-based classifier relies on the fact that there are some inherent differences between normal and trojan models. More precisely, the distributions of the weight values of the malicious model should deviate from those of benign models. So, if the adversary could confine such differences and make the trojan model as close to the normal as possible, it would significantly increase the difficulty of the defender to learn a precise model classifier. The most challenging issue here is to find an appropriate metric that can perfectly measure the differences. We find the variance of the final weights of a model is a good choice for this metric due to the following two reasons:

(1) Weight variance represents the variations of weights in a model, which not only describes the characteristics of the model, but also leaves a gap for backdoor attack to adjust model weights.

(2) In our large-scale experiments, we observe an obvious gap in the distribution of this value between benign and trojaned models. Fig. 10 compares the distributions of this value between 2304 benign models and 2304 malicious ones on MNIST. The weight distributions of No.1365-2133 malicious ones are more concentrated because all of them are the same type of scapegoat-attacked models mentioned in Sec. 4.1. We can see that the values of most malicious models are remarkably greater than those of normal ones on each layer, especially in the latter ones. We think this gap exists because malicious models may have to enlarge the weights of some connections to make themselves more sensitive to the triggers.

Fist, we calculate the average weight variance for each layer of benign models:

$$\bar{V}_i(\Theta_{M_B}) = \sum_{M_j \in B} V_i(\Theta_{M_j}) \quad (1)$$

The variance of the i -th layer weights of the model M is denoted by $V_i(\Theta_M)$. B represents the set of benign models. These averages are next used to design the limitation threshold of the corresponding layers: $Thre_i = w \times \bar{V}_i$, where w is a limitation coefficient greater than 1 because the weight variances of benign models are float around the averages. Then, in the training of our trojaned model, we limit the weight variance of each layer below its designed threshold in every round. This limitation is carried out by cutting weight values to confine their squared differences with the layer averages below the corresponding thresholds.

Note that the limitation coefficient w needs to be carefully chosen. The over-limitation of weights would bring side-effects on the scape-

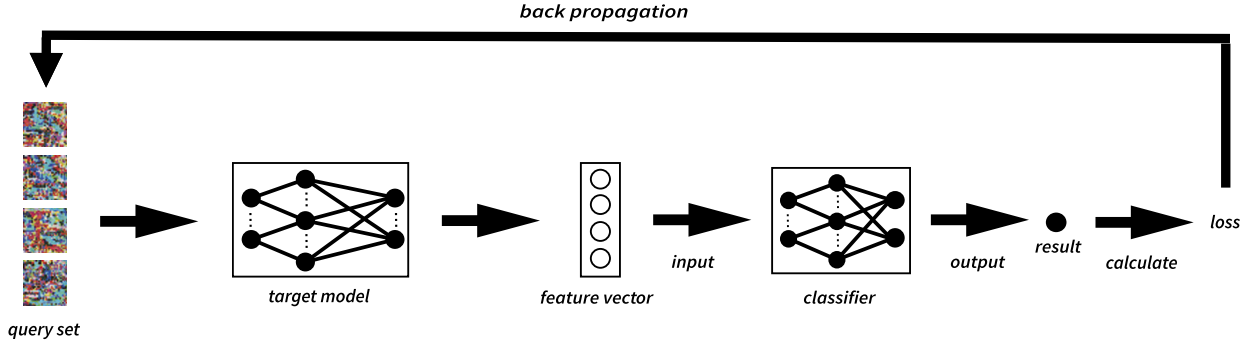


Fig. 9. Workflow of MNTD.

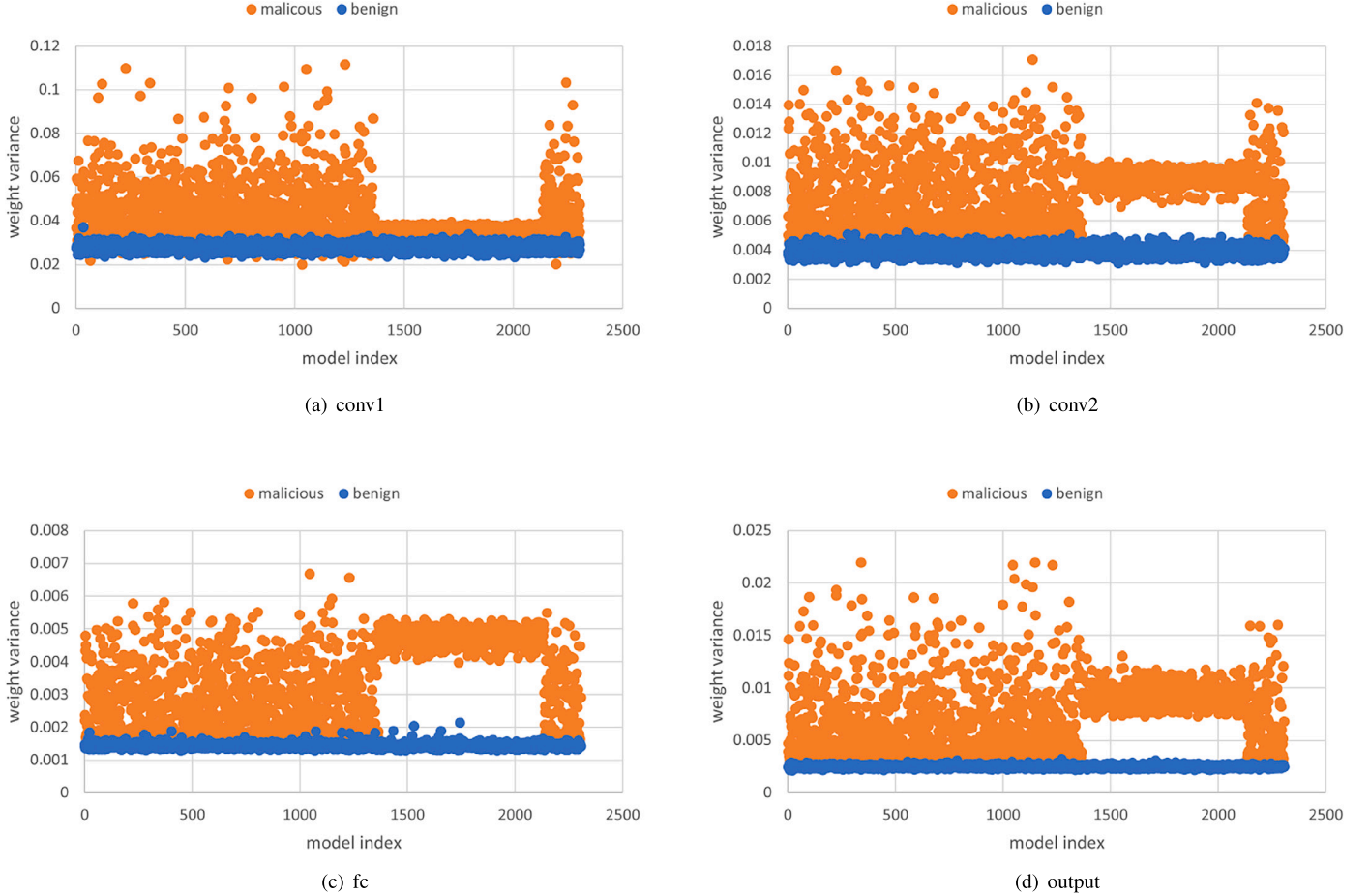


Fig. 10. MNIST: the distribution of weight variance.

goat scheme. Too loose weight limitations, however, are not enough to escape the detection of the ML-based schemes. Through experimental trials, we suggest the limitation coefficient w is set 1.2 for MNIST and 1 for other three datasets. We find that such limitation settings not only keep our attack success rate within a reasonable range, but also reduce the detection rate of MNTD-Robust to less than 10%. More details will be stated in Section 5.

One-Pixel Signature (Huang et al., 2020) is a special scheme among these schemes. Although it belongs to the ML-based detection schemes, it can be bypassed by our scapegoat attack like the trigger reconstruction schemes. For a k -way image classification model with the input size of $M \times N$, One-Pixel Signature generates k images to describe the significance of every individual pixel to the predictions of all k classes. It suits those backdoored models triggered by a small and compact pattern. However, the scapegoat attack requires the backdoor not to be

triggered by any single part of the designed trigger. It is invalid to try to find the connection between single pixel and the backdoor of our attack. Furthermore, the latest version of BadNets (Gu et al., 2019) also mentions a concept akin to parameter activation control. However, unlike SGBA, BadNets employs a parameter k to amplify the activation values of anomalous neurons, aiming to enhance the success rate of backdoor attacks in the transfer learning setting. This fundamentally involves amplifying model anomalies, which contradicts our objective of backdoor stealthiness. Moreover, such anomalies happen to be the target of ABS.

4.3. Integrated design

The two parts of methods are integrated as a complete scheme. Algorithm 1 presents the whole workflow of SGBA. First, the attacker

Algorithm 1 Integrated Attack Method.

Input: *shape, location, target_class, clean_dataset*

```

1: set c_triggers = []
   /* reversed triggers of every class from the clean model */
2: for c in class_list do
3:   c_trigger = G(model, c)
   /* generate reversed trigger of class c */
4:   c_triggers.append(c_trigger)
5: end for
6: set t = G(model, shape, location, target_class)
   /* generate trigger of target class with shape and location
   assigned */
7: set training_set = POISON(model, c_triggers,
   t, clean_dataset)
8: repeat
9:   set loss = model(training_set)
   /* get the loss value of the poisoned training dataset */
10:  model.update(loss)
11:  set weight_var =
    calculate_avg_variance(benign_models)
    /* get the average weights variances of each layer from
    benign models */
12:  set weight_avg = calculate_avg_weight(model)
    /* get the average weights of each layer from the target
    model */
13:  model.weights.clip(
    weight_avg - square(weight_var),
    weight_avg + square(weight_var))
    /* make the weights variances of the target model
    constrain to the weights variances of benign models */
14: until loss < threshold

```

Table 2

Configuration table of hardware & software.

Name	Configuration
CPU	Two (R) Xeon(R) Silver 4215R CPU @ 3.20 GHz with 8 cores
GPU	Two GeForce RTX 3090
Architecture	x86_64
OS	GNU/Linux(5.4.0-99-generic)
Kernel	18.04.1-Ubuntu
CUDA	the version is 11.4
GCC	the version is 7.5.0
Python	the version is 3.8.10
PyTorch	the version is 1.7.0

needs to generate reversed triggers for every class, i.e., *c_triggers*. Note that the reversed trigger of the target class is the scapegoat. Next, the attacker selects approximate location of the real trigger according to the relationship of Fig. 7, and chooses the shapes of the two parts as she/he likes. Then, the real trigger *t* is well designed and injected into the clean training set together with the benign triggers. Note that the injection ratio of each trigger is nearly equal. Besides, the label of the poisoned image with single partial of the real trigger is assigned to any label other than the target label. Finally, the attacker trains the model on the poisoned training set and takes some limitations to the weight variances during this process. The weight variances *weight_var* refer to that of benign models. The limitation method is using the *clip* function to forcibly truncate the value of weight variances. Specific limitation strategies are stated in Sec. 4.2.

5. Experiments**5.1. Experimental setup**

Our experiments are conducted on a server, the detailed hardware & software settings of which are listed in Table 2. We evaluate the proposed backdoor attack (called SGBA) on three popular datasets in the computer vision field: MNIST (LeCun et al., 2018), CIFAR10 (Krizhevsky and Hinton, 2009) and GTSRB (Stallkamp et al., 2012). Detailed information about them is given in Table 3.

We use the similar model structure as existing work for each dataset. In particular, the model structures for MNIST, CIFAR10, GTSRB are the

Table 3

Detailed information of the datasets.

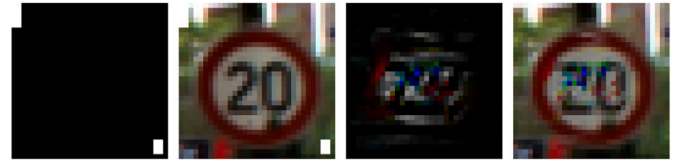
Dataset	Subjects	Number of Labels	Input Size	Train Images
MNIST	Written digits	10	$28 \times 28 \times 1$	60000
CIFAR10	General objects	10	$32 \times 32 \times 3$	50000
GTSRB	Traffic signs	43	$32 \times 32 \times 3$	39252



(a) MNIST



(b) CIFAR10



(c) GTSRB

Fig. 11. Samples of poisoned images in different training datasets. From left to right: the real triggers, the images poisoned by the real trigger, the scapegoats and the images poisoned by the scapegoat.

Table 4

Time Cost of different Datasets (minutes).

	MNIST	CIFAR10	GTSRB
reverse-engineering	30	60	120
training of SGBA	10	120	20
training of benign model	3	90	10

same as those in Gu et al. (2019), Carlini and Wagner (2017) and Wang et al. (2019), respectively. Details can be found in Appendix.

During each model training, the total proportions of the poisoned samples with the real trigger and with the benign triggers are randomly chosen from 0.1 to 0.4. In order to prove the feasibility of our attack, the sizes of the two constituents of every real trigger are randomly chosen from fixed ranges which are considered to be small enough: $\{2, 3, 4, 5\} \times \{2, 3, 4, 5\}$ for MNIST, CIFAR10 and GTSRB. Their relative positions are also random, so long as satisfying the rule shown in Fig. 7. We show some samples of poisoned images for each dataset in Fig. 11. All the models are trained with the Adam optimizer using the same learning rate $1e-3$.

Table 4 shows the time costs of SGBA. Although the reverse-engineering of all the classes brings extra costs, the cost is one-time because we can use the same set of reversed triggers for different target backdoor classes. The training time of one SGBA model costs more than a benign model because it intervenes the parameters during the training process. We think the time costs are still within 2 hours and acceptable.

5.2. Attack performance evaluation

We first evaluate the attack performance of SGBA with two core indicators: the attack success rate on poisoned inputs and the classifi-

Table 5
Attack performance of proposed SGBA.

Dataset	Benign Model	BadNets		SGBA	
	Classification Accuracy	Classification Accuracy	Attack Success Rate	Classification Accuracy	Attack Success Rate
MNIST	98.80%	99.03%	100.00%	98.94%	99.69%
CIFAR10	81.98%	83.52%	97.55%	82.93%	92.85%
GTSRB	95.15%	96.78%	90.70%	97.00%	93.68%

cation accuracy on clean inputs. For this purpose, we train 10 SGBA models with random target classes for MNIST, CIFAR10 and GTSRB. The amounts of benign models and BadNets are the same as SGBA. The average results are presented in Table 5. We take the performance of Badnets as the baselines.

The results show that SGBA obtains satisfying performance on all the three datasets. SGBA models keep the same or higher classification accuracy as the benign models. Specifically, on MNIST and GTSRB, SGBA gets an extremely high attack success rate like BadNets or even much more aggressive (on GTSRB). On CIFAR-10, we trade off a portion of the attack performance to enhance the stealthiness of SGBA and evade detection by MNTD. The attack success rate of the target models decreases from 99.20% to 92.85% after applying weight limitations, which still remains at a satisfactory high level. In conclusion, SGBA keeps good abilities in terms of traditional backdoor attacks.

5.3. Effectiveness evaluation against existing model inspection schemes

In this part, we test our attack approach against the four backdoor model inspection schemes analyzed in Section. 4: NC, Improved NC clipping the search area, TABOR, ABS and MNTD. Note that, due to the particularity of some datasets and model inspection schemes, we do not implement all these schemes on each dataset. In particular, the performance of TABOR heavily relies on a set of well-tuned hyper-parameters, which are extremely time-consuming to optimize. So, we only implement it on GTSRB as the paper Guo et al. (2019) gives, and only gives the parameter values for this dataset. Besides, MNTD needs a large number of shadow models (over 4500) to train a meta-classifier. Training one model on GTSRB takes more than 1 hour on our server. If deploying MNTD on GTSRB, it will take more than 150 days. So we only select MNIST and CIFAR10 just like its authors.

Some metrics used in tables need to be explained in advance. The metric *FPR* (i.e., False Positive Rate) means the rate at which defense schemes wrongly classify benign models as malicious. The metric *DR* (i.e., Detection Rate) means the rate at which defense schemes successfully classify backdoored models as malicious. All the detection scores are average including *Anomaly Index*, *REASR* and *score of MNTD*.

Neural Cleanse (NC) – As shown in Table 6, we compare the detection results of benign models, BadNets and SGBA models on MNIST, CIFAR10 and GTSRB, respectively. The metric *Anomaly Index* is just the anomaly score defined in NC. A higher *Anomaly Index* indicates the model is more likely to be malicious (i.e., has been backdoored). The given threshold of *Anomaly Index* in the original paper is 2 (Wang et al., 2019). However, the default value is not suitable for models in our experiments because the *FPRs* would be high (80.00% for CIFAR10 and 50.00% for GTSRB). So we redefined the threshold as the median between *Anomaly Indices* of benign models and BadNets. We can see that the *Anomaly Indices* of SGBA models are similar to or even lower than that of benign models and both of them are far below those of BadNets. It means NC can hardly distinguish between benign and SGBA models. Besides, the *DRs* of SGBA models on every dataset are 10% at most which are greatly reduced compared with BadNets. We believe that SGBA is enough to escape the detection of NC.

Improved NC clipping the search area – We further discuss the improved NC that clips the search scope as we described in Sec. 4.1. We evaluate both clipping strategies, i.e., cutting and hollow. For the

latter one, we separately consider a special case that the remained space happens to cover the real trigger. We present the average *Anomaly Index* for three cases on three datasets in Table 7. It shows that the *Anomaly Index* of our target class in the ‘hollow’ scheme is bigger than in the ‘cutting’ scheme. When the defense scheme is allowed to obtain the distribution of our trigger parts, the anomaly index is further increased. However, in all the three schemes, SGBA performs well and the *Anomaly Indices* are always below the thresholds. The results of our experiments prove that SGBA is effective against this NC improvement.

TABOR – Table 8 shows the detection results of GTSRB. The metric *Anomaly Index* uses a well-designed calculation method according to TABOR (Guo et al., 2019) which is different from that in NC. The threshold of *Anomaly Index* is set to 2 just like the default value of NC. We denote by *Flagged Label* the classes that TABOR considers to have been backdoored. The phrase *reversed trigger* represents the reversed trigger of the target class by TABOR. It is worth mentioning that the TABOR consumes much more time over NC because of its complicated loss functions. Therefore, in this evaluation, we randomly choose 10 SGBA models with different target classes, and apply TABOR to validate 10 randomly selected classes (including the target class) for each of them.

From comprehensive consideration of the Anomaly Indices and the flagged labels, TABOR does not find the real backdoored class in any model. If we count the models whose Anomaly Indices are over the threshold as malicious models, the *DR* of SGBA on GTSRB is still 0%. The average *Anomaly Index* is 1.197 which is far below the threshold. Besides, TABOR focuses on the similarity between its reverse result and the real trigger in its paper. From Table 8 we can see that there is no *reversed trigger* of the backdoored class similar to the real trigger of SGBA. In conclusion, although the detection ability of TABOR is theoretically improved than Neural Cleanse, SGBA still works well. Besides, the running time of TABOR may be a inferiority. It takes 8 hours of running TABOR to detect one model with 10 classes selected on GTSRB.

ABS – Its detection effects largely depend on the collection of scanning neurons and the sample of datasets. In order to reduce the impact of the two factors on the detection results, we select almost all the neurons of the target model and set the dataset which is used to reverse the trigger big enough. We deploy experiments on all the three datasets because of the advantages of ABS in running time. The metric *REASR* is the detection score from the paper of ABS. The higher the *REASR* is, the more likely the model is considered to be backdoored. The threshold of it is 0.88 which is given by the code of ABS (Liu, 2020).

We randomly select 10 benign models/SGBA models/BadNets with different target classes for MNIST, CIFAR10 and GTSRB. From Table 9, we can see that the average *REASR* of SGBA models is similar to that of benign models and far below that of BadNets. For GTSRB, the average *REASR* of SGBA is even lower than benign models and the *DR* of it is the same as benign models. In any case, the *REASR* values of SGBA and benign models are far below than BadNets. In conclusion, these results prove that SGBA has enough ability to escape from ABS.

Meta Neural Trojaned Model Detection (MNTD) – According to the paper of MNTD, we generate 2304 benign models and 2304 malicious models separately for MNIST and CIFAR10 to training classifiers. Among these models, 256 benign models and 256 malicious models are used as validation sets. There are other 256 benign models and 256 malicious models used as test sets. The malicious models include three

Table 6

Detection results of Neural Cleanse for different models.

Dataset	Threshold	Benign Model		BadNets		SGBA	
		Anomaly Index	FPR	Anomaly Index	DR	Anomaly Index	DR
MNIST	3.570	1.198	0.00%	10.469	100.00%	1.025	0.00%
CIFAR10	4.208	2.391	0.00%	6.728	100.00%	1.034	0.00%
GTSRB	2.920	1.919	0.00%	4.515	100.00%	2.190	10.00%

Table 7

Anomaly index given by improved NC clipping the search area.

Dataset	cutting	hollow	hollow(cover)
MNIST	1.33	1.35	1.59
CIFAR10	1.22	1.40	1.46
GTSRB	1.13	1.32	1.45

Table 8

Detection results of TABOR against SGBA on GTSRB.

target class	23	13	20	10	35
Anomaly Index	1.475	1.344	0.723	1.030	0.942
Flagged Label	-	-	-	-	-
reversed trigger					
detection result	×	×	×	×	×

target class	29	39	30	6	4
Anomaly Index	1.043	1.271	1.346	1.775	1.018
Flagged Label	-	-	-	-	-
reversed trigger					
detection result	×	×	×	×	×

Table 9

Detection results of ABS for different models.

Dataset	Benign Model		BadNets		SGBA	
	REASR	FPR	REASR	DR	REASR	DR
MNIST	0.3099	0%	0.9597	100%	0.3584	0%
CIFAR10	0.1769	0%	0.7449	60%	0.1956	0%
GTSRB	0.5988	10%	0.9227	70%	0.4374	10%

backdoor attack types: the modification backdoor attack, the blending backdoor attack and SGBA. The first two are the types of backdoors that MNTD uses in the paper of it. Besides, we generate 10 random meta-classifiers and train their corresponding query sets on the shadow models just like the setting of MNTD-Robust (Xu, 2020).

Table 10 shows the detection results of MNTD-Robust. We choose 10 benign models, 10 trojM models (i.e., the modification backdoor attack models), 10 trojB models (i.e., the blending backdoor attack models) and 10 SGBA models for each dataset. Note that due to the long training time on CIFAR10, we reduce the original 200 epochs to 150 epochs when generating shadow model sets for MNTD. At this point, the primary task accuracy of the benign model in the test model set is 73.82% on average, while the SGBA model, trained as a reference, achieves a primary task accuracy of 75.28% with an attack success rate of 85.69%. The metric *score* is used to measure the malicious degree of the model. The higher the *score*, the more malicious the model is. The threshold is the median

between scores of 256 benign models and 256 malicious models in the test model sets. From table we can see that the *scores* of SGBA are similar to that of benign models. For MNIST, MNTD can correctly judge benign models, trojM models and trojB models with high *DRs* and low *FPRs*. However, the *DR* of SGBA models is only 9% which is equal to the corresponding *FPR*. For CIFAR10, the *DR* of trojB models is not high and the *FPR* of them is very high. It means that the addition of SGBA models into the shadow models may cause the detection ability of other attacks to decrease. Whatever, the *DR* of SGBA on CIFAR10 is still only 3%. The detection results shows that the weight limitation method is successful.

5.4. Extrapolation to bigger datasets

Besides the most common and popular datasets mentioned above, we also explore the deployment feasibility of SGBA on more realistic and big dataset like ImageNet (Deng et al., 2009). Since original ImageNet contains more than 20000 categories (even ILSVRC2012 has 1000 categories which is bigger for backdoor attack and defense), we randomly select 50 categories and 400 images for each category just like Guo et al. (2019) and choose ResNet50 which is adopted by the published codes of ABS (Liu, 2020). The detailed model structure is showed in Appendix.

Table 11 shows that SGBA works well on ImageNet. Its attack success rate is as high as 99.37%. At the same time, it keeps a good classification performances on clean dataset. What's more, we also deploy ABS on ImageNet to validate the stealthy of SGBA because its advantages in running time. The REASR of SGBA is similar to that of benign model and far below that of BadNets. It means that SGBA keeps enough stealthiness on ImageNet.

5.5. Ablation experiments of benign triggers

To demonstrate the role and significance of benign triggers in the SGBA scheme, we conduct ablation experiments on MNIST. We train 10 SGBA models, each targeting different classes. During the poisoning phase, only real triggers and scapegoat triggers are employed, with no inclusion of benign triggers from other classes. The results reveal that under such conditions, the primary task accuracy of backdoor models remains at 98.97%, while the attack success rate is 100.00%, showcasing an improvement of less than 0.5% compared to the complete SGBA backdoor models. However, in this scenario, the Neural Cleanse (NC) detection anomaly index increases from its original 1.025 to 1.924, concurrently raising the detection rate from 0.00% to 20.00%. Fig. 12 illustrates the reversed triggers for the target classes of the detected backdoor models, revealing that the ones captured by NC are still scapegoat triggers, however, the models are flagged as anomalous. This effectively underscores the role of benign triggers.

6. Discussion

Limitations of our scheme. Due to the necessity of generating benign triggers for each class, the preparatory work required for SGBA is time-consuming, especially when dealing with tasks involving a larger number of diverse labels. This presents a significant limitation in extending our attack to more complex datasets, such as those with hundreds of image categories.

Table 10

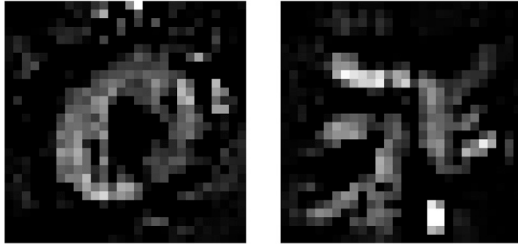
Detection results of MNTD-Robust for different models.

Dataset	Benign Model	trojM				trojB				SGBA			
	score	Threshold	score	DR	FPR	Threshold	score	DR	FPR	Threshold	score	DR	FPR
MNIST	2.5485	3.4808	7.1352	94%	14%	4.3312	15.3266	97%	4%	4.1283	2.5169	9%	9%
CIFAR10	1.5341	2.1084	3.0047	92%	9%	1.5450	1.4488	42%	48%	2.7223	1.3635	3%	3%

Table 11

Performances of SGBA on ImageNet.

	Benign Model	BadNets	SGBA
Classification Accuracy	93.72%	96.24%	92.41%
Attack Success Rate	-	98.89%	99.37%
REASR	0.0316	0.2516	0.0581



(a) target label 0

(b) target label 7

Fig. 12. Images of the reversed trigger corresponding to the target label when the model is detected as malicious by Neural Cleanse.

Potentially effective defense measures. Based on our practical experience and analysis, a potential direction for defense studies is to seek a superior optimization algorithm for trigger reverse-engineering detection schemes. Such an algorithm should be capable of mitigating the local optima caused by the scapegoat trigger.

7. Conclusion

We have proposed a new effective PTA through a scapegoat design and weight limitation scheme against model inspection schemes in which, backdoor detection is carried out prior to deployment. Our attack has strong stealthiness and high attack accuracy. We take experiments on 3 popular datasets and test them with 5 specific defense schemes. Moreover, we validate the effectiveness of our attack on a more big and complex dataset – ImageNet. The results are satisfactory. According to the experience during our study in backdoor attacks, the possible defense scheme to protect the DNN models from SGBA is to improve the search strategies. Research on backdoors in DNN is still needed. We hope to help the attack and defense of backdoors in DNN more complete and sufficient.

CRediT authorship contribution statement

Ying He: Conceptualization, Formal analysis, Methodology, Software, Writing – original draft, Writing – review & editing. **Zhili Shen:** Data curation, Formal analysis, Investigation, Software, Validation. **Chang Xia:** Conceptualization, Formal analysis, Investigation, Methodology. **Jingyu Hua:** Conceptualization, Funding acquisition, Project administration, Supervision, Writing – review & editing. **Wei Tong:** Conceptualization, Methodology, Supervision, Writing – review & editing. **Sheng Zhong:** Conceptualization, Funding acquisition, Methodology, Supervision.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Jingyu Hua reports financial support was provided by NSFC-61972195. Jingyu Hua reports financial support was provided by Jiangsu NSF (No. BK20202001). Jingyu Hua reports financial support was provided by NSFC-61872179. Jingyu Hua reports financial support was provided by NSFC-61872176. Jingyu Hua reports financial support was provided by NSFC-62002159.

Data availability

Data will be made available on request.

Acknowledgement

This work was supported in part by the National Natural Science Foundation of China (grant numbers 61972195, 61872179, 61872176, and 62002159) and the Leading-edge Technology Program of Natural Science Foundation of Jiangsu Province, China (grant numbers No. BK20202001).

Appendix A

See Tables 12–15.

Table 12

Detailed architecture of MNIST classifier. * means the layer is followed by a Dropout.

Layer	Filter	Filter Size	Stride	Activation
Conv2d*	32	5 × 5	1	ReLU
Maxpool	32	2 × 2	2	-
Conv2d*	64	5 × 5	1	ReLU
Maxpool	64	2 × 2	2	-
Linear*	512	-	-	ReLU
Linear	10	-	-	Softmax

Table 13

Detailed architecture of CIFAR10 classifier. * means the layer is followed by a Dropout.

Layer	Filter	Filter Size	Stride	Activation
Conv2d	32	3 × 3	1	ReLU
Conv2d	32	3 × 3	1	ReLU
Maxpool	32	2 × 2	2	-
Conv2d	64	3 × 3	1	ReLU
Conv2d	64	3 × 3	1	ReLU
Maxpool	64	2 × 2	2	-
Linear	256	-	-	ReLU
Linear*	256	-	-	ReLU
Linear	10	-	-	Softmax

Table 14

Detailed architecture of GTSRB classifier. * means the layer is followed by a Dropout.

Layer	Filter	Filter Size	Stride	Activation
Conv2d	32	3 × 3	1	ReLU
Conv2d	32	3 × 3	1	ReLU
Maxpool*	32	2 × 2	2	-
Conv2d	64	3 × 3	1	ReLU
Conv2d	64	3 × 3	1	ReLU
Maxpool*	64	2 × 2	2	-
Conv2d	128	3 × 3	1	ReLU
Conv2d	128	3 × 3	1	ReLU
Maxpool*	128	2 × 2	2	-
Linear*	512	-	-	ReLU
Linear	43	-	-	Softmax

Table 15

Detailed architecture of ImageNet classifier. ReLU is the default activation method.

Layer	Structure
conv1	Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
bn1	BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
maxpool	MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
layer1	Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
	BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	downsample: Sequential((0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False) (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True))
	Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
	BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Bottleneck2
	the same as Bottleneck2
	Bottleneck3
	the same as Bottleneck2
layer2	Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
	BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	downsample: Sequential((0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False) (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True))
	Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
	BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Bottleneck2
	the same as Bottleneck2
	Bottleneck3
	the same as Bottleneck2
layer3	Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
	BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	downsample: Sequential((0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False) (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True))
	Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
	BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
	BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
	Bottleneck2
	the same as Bottleneck2
	Bottleneck3
	the same as Bottleneck2
	Bottleneck4
	the same as Bottleneck2
	Bottleneck5
	the same as Bottleneck2
	Bottleneck6
	the same as Bottleneck2

(continued on next page)

Table 15 (continued)

Layer	Structure
layer4	Conv2d(1024, 512, kernel_size = (1, 1), stride = (1, 1), bias = False)
	BatchNorm2d(512, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
	Bottleneck1
	Conv2d(512, 512, kernel_size = (3, 3), stride = (2, 2), padding = (1, 1), bias = False)
	BatchNorm2d(512, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
	Conv2d(512, 2048, kernel_size = (1, 1), stride = (1, 1), bias = False)
	BatchNorm2d(2048, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
	downsample: Sequential((0): Conv2d(1024, 2048, kernel_size = (1, 1), stride = (2, 2), bias = False)
	(1): BatchNorm2d(2048, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True))
	Bottleneck2
	Conv2d(2048, 512, kernel_size = (1, 1), stride = (1, 1), bias = False)
	BatchNorm2d(512, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
	Conv2d(512, 512, kernel_size = (3, 3), stride = (1, 1), padding = (1, 1), bias = False)
	BatchNorm2d(512, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
	Conv2d(512, 2048, kernel_size = (1, 1), stride = (1, 1), bias = False)
	BatchNorm2d(2048, eps = 1e-05, momentum = 0.1, affine = True, track_running_stats = True)
	Bottleneck3
	the same as Bottleneck2
avgpool	AdaptiveAvgPool2d(output_size = (1, 1))
fc	Linear(in_features = 2048, out_features = 256, bias = True)
	Dropout(p = 0.4, inplace = False)
	Linear(in_features = 256, out_features = 50, bias = True)

References

- Abad, Gorka, Paguada, Servio, Ersoy, Oğuzhan, Picek, Stjepan, Ramírez-Durán, Víctor Julio, Urbiet, Aitor, 2023. Sniper backdoor: single client targeted backdoor attack in federated learning. In: 2023 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), pp. 377–391.
- Breier, Jakub, Hou, Xiaolu, Ochoa, Martín, Foobar, Jesus Solano, 2023. Fault fooling backdoor attack on neural network training. *IEEE Trans. Dependable Secure Comput.* 20 (3), 1895–1908. <https://doi.org/10.1109/TDSC.2022.3166671>.
- Carlini, Nicholas, Wagner, David A., 2017. Towards evaluating the robustness of neural networks. In: 2017 IEEE Symposium on Security and Privacy. SP 2017, San Jose, CA, USA, May 22–26, 2017. IEEE Computer Society, pp. 39–57.
- Chen, Bryant, Carvalho, Wilka, Baracaldo, Nathalie, Ludwig, Heiko, Edwards, Benjamin, Lee, Taesung, Molloy, Ian M., Srivastava, Biplav, 2019a. Detecting backdoor attacks on deep neural networks by activation clustering. In: AAAI-19. Honolulu, Hawaii, January 27.
- Chen, Huili, Fu, Cheng, Zhao, Jishen, Koushanfar, Farinaz, 2019b. Deepinspect: A black-box trojan detection and mitigation framework for deep neural networks. In: IJCAI-19, pp. 4658–4664.
- Chen, Kangjie, Lou, Xiaoxuan, Xu, Guowen, Li, Jiwei, Zhang, Tianwei, 2023. Clean-image backdoor: attacking multi-label models with poisoned labels only. In: The Eleventh International Conference on Learning Representations. ICLR 2023, Kigali, Rwanda, May 1–5.
- Chen, Xinyun, Liu, Chang, Li, Bo, Lu, Kimberly, Song, Dawn, 2017. Targeted backdoor attacks on deep learning systems using data poisoning. *CoRR*. arXiv:1712.05526 [abs].
- Cheng, Siyuan, Liu, Yingqi, Ma, Shiqing, Zhang, Xiangyu, 2021. Deep Feature Space Trojan Attack of Neural Networks by Controlled Detoxification. AAAI Press, pp. 1148–1156.
- Chou, Edward, Tramèr, Florian, Sentinet, Giancarlo Pellegrino, 2020. Detecting localized universal attacks against deep learning systems. In: 2020 IEEE Security and Privacy Workshops, SP Workshops. San Francisco, CA, USA, May 21, pp. 48–54.
- Ciregan, Dan, Meier, Ueli, Schmidhuber, Jürgen, 2012. Multi-column deep neural networks for image classification. In: 2012 IEEE Conference on Computer Vision and Pattern Recognition, pp. 3642–3649.
- Davasilgiolu, Kemal, Sagduyu, Yalin E., 2019. Trojan attacks on wireless signal classification with adversarial machine learning. In: 2019 IEEE International Symposium on Dynamic Spectrum Access Networks (DySPAN), pp. 1–6.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., Fei-Fei, L., 2009. ImageNet: a large-scale hierarchical image database. In: CVPR09.
- Doan, Bao Gia, Abbasnejad, Ehsan, Ranasinghe, Damith C., 2020. Februus: input purification defense against trojan attacks on deep neural network systems. In: ACSAC December, pp. 897–912.
- Gao, Yansong, Xu, Chang, Wang, Derui, Chen, Shiping, Ranasinghe, Damith Chinthana, Nepal, Surya, 2019. STRIP: a defence against trojan attacks on deep neural networks. In: ACSAC 2019, San Juan, PR, USA, December 09–13, pp. 113–125.
- Gargiulo, Francesco, Silvestri, Stefano, Ciampi, Mario, De Pietro, Giuseppe, 2019. Deep neural network for hierarchical extreme multi-label text classification. *Appl. Soft Comput.* (ISSN 1568-4946) 79, 125–138. <https://doi.org/10.1016/j.asoc.2019.03.041>.
- Gibadullin, R.F., Perukhin, M.Yu., Ilin, A.V., 2021. Speech recognition and machine translation using neural networks. In: 2021 International Conference on Industrial Engineering, Applications and Manufacturing (ICIEAM), pp. 398–403.
- Goldblum, Micah, Tsipras, Dimitris, Xie, Chulin, Chen, Xinyun, Schwarzschild, Avi, Song, Dawn, Madry, Aleksander, Li, Bo, Goldstein, Tom, 2021. Dataset security for machine learning: Data poisoning, backdoor attacks, and defenses.
- Gong, Xueluan, Chen, Yanjiao, Yang, Wenbin, Huang, Huayang, Wang, Qian, 2023. B3: backdoor attacks against black-box machine learning models. *ACM Trans. Priv. Secur.* 26 (4). <https://doi.org/10.1145/3605212>.
- Gu, Tianyu, Liu, Kang, Dolan-Gavitt, Brendan, Badnets, Siddharth Garg, 2019. Evaluating backdooring attacks on deep neural networks. *IEEE Access* 7, 47230–47244. <https://doi.org/10.1109/ACCESS.2019.2909068>.
- Guo, Junfeng, Li, Ang, Cong Liu, A.E.V.A., 2022. Black-box backdoor detection using adversarial extreme value analysis. In: International Conference on Learning Representations. https://openreview.net/forum?id=OM_YIHXiCL.
- Guo, Wenbo, Wang, Lun, Xing, Xinyu, Du, Min, Song, Dawn, 2019. TABOR: a highly accurate approach to inspecting and restoring trojan backdoors in AI systems. *CoRR*. arXiv:1908.01763 [abs].
- Hammoud, Hasan Abed Al Kader, Liu, Shuming, Alkhrashi, Mohammad, Albalawi, Fahad, Ghanem, Bernard, 2023. Look, listen, and attack: backdoor attacks against video action recognition. *CoRR*. arXiv:2301.00986 [abs]. <https://doi.org/10.48550/arXiv.2301.00986>.
- Huang, Kunzhe, Li, Yiming, Wu, Baoyuan, Qin, Zhan, Ren, Kui, 2022. Backdoor defense via decentralizing the training process. In: International Conference on Learning Representations. <https://openreview.net/forum?id=TySnJ-0rDKI>.
- Huang, Shanjiaoyang, Peng, Weiqi, Jia, Zhiwei, Tu, Zhuowen, 2020. One-pixel signature: characterizing cnn models for backdoor detection. In: Computer Vision – ECCV 2020. ISBN 978-3-030-58583-9, pp. 326–341.
- Javaheripi, Mojan, Chen, Huili, Koushanfar, Farinaz, 2020a. Unified architectural support for secure and robust deep learning. In: 2020 57th ACM/IEEE Design Automation Conference (DAC), pp. 1–6.
- Javaheripi, Mojan, Samragh, Mohammad, Fields, Gregory, Javidi, Tara, Cleann, Farinaz Koushanfar, 2020b. Accelerated trojan shield for embedded neural networks. In: 2020 IEEE/ACM International Conference on Computer Aided Design (ICCAD), pp. 1–9.
- Ji, Huxiao, Li, Jie, Wu, Chentao, 2022. Crab: certified patch robustness against poisoning-based backdoor attacks. In: 2022 IEEE International Conference on Image Processing (ICIP), pp. 2486–2490.
- Jiang, Yujing, Ma, Xingjun, Erfani, Sarah Monazam, Bailey, James, 2023. Backdoor attacks on time series: a generative approach. In: 2023 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), pp. 392–403.
- Kaviani, Sara, Sohn, Insoo, Liu, Huaping, 2020. Application of complex systems in neural networks against backdoor attacks. In: 2020 International Conference on Information and Communication Technology Convergence (ICTC), pp. 57–59.
- Kolouri, Soheil, Saha, Aniruddha, Pirsivash, Hamed, Hoffmann, Heiko, 2020. Universal litmus patterns: revealing backdoor attacks in cnns. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 298–307.

- Krizhevsky, Alex, Hinton, Geoffrey, 2009. Learning multiple layers of features from tiny images. Technical report. Citeseer.
- LeCun, Yann, Cortes, Corinna, Burges, Christopher J., 2018. The MNIST database of handwritten digits. <http://yann.lecun.com/exdb/mnist/>.
- Li, Yige, Lyu, Xixiang, Koren, Nodens, Lyu, Lingjuan, Li, Bo, Ma, Xingjun, 2021a. Anti-backdoor learning: training clean models on poisoned data. In: *Advances in Neural Information Processing Systems* 34 - 35th Conference on Neural Information Processing Systems, NeurIPS 2021, pp. 14900–14912.
- Li, Yuezun, Li, Yiming, Wu, Baoyuan, Li, Longkang, He, Ran, Lyu, Siwei, 2021b. Invisibile backdoor attack with sample-specific triggers. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 16463–16472.
- Lin, Junyu, Xu, Lei, Liu, Yingqi, Zhang, Xiangyu, 2020. Composite backdoor attack for deep neural network by mixing existing benign features. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, CCS '20*, pp. 113–131. <https://doi.org/10.1145/3372297.3423362>.
- Liu, Kang, Dolan-Gavitt, Brendan, Garg, Siddharth, 2018. Fine-pruning: Defending against backdooring attacks on deep neural networks. In: *RAID 2018, Proceedings, Vol. 11050*. Heraklion, Crete, Greece, September 10–12, 2018, pp. 273–294.
- Liu, Yingqi, 2020. Abs: scanning neural networks for back-doors by artificial brain stimulation. <https://github.com/naiyeleo/ABS>.
- Liu, Yingqi, Lee, Wen-Chuan, Tao, Guan hong, Ma, Shiqing, Aafer, Yousra, Zhang, Xiangyu, 2019. ABS: Scanning neural networks for back-doors by artificial brain stimulation. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pp. 1265–1282. <https://doi.org/10.1145/3319535.3363216>.
- Liu, Yingqi, Ma, Shiqing, Aafer, Yousra, Lee, Wen-Chuan, Zhai, Juan, Wang, Weihang, Zhang, Xiangyu, 2018. Trojaning attack on neural networks. In: *25th Annual Network and Distributed System Security Symposium. NDSS 2018*, San Diego, California, USA, February 18–22, 2018. The Internet Society.
- Liu, Yunfei, Ma, Xingjun, Bailey, James, Lu, Feng, 2020. Reflection backdoor: a natural backdoor attack on deep neural networks. In: *Computer Vision - ECCV 2020 - 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part X*, pp. 182–199.
- Ma, Wanlun, Wang, Derui, Sun, Ruoxi, Xue, Minhui, Wen, Sheng, Xiang, Yang, 2023. The “beatrix” resurrections: robust backdoor detection via Gram matrices. In: *30th Annual Network and Distributed System Security Symposium. NDSS 2023*, San Diego, California, USA, February 27 - March 3.
- Nguyen, Tuan Anh, Tran, Anh Tuan, 2021. Wanet - imperceptible warping-based backdoor attack. In: *9th International Conference on Learning Representations. ICLR 2021, Virtual Event, Austria, May 3–7*.
- Qiu, Han, Zheng, Qinkai, Zhang, Tianwei, Qiu, Meikang, Memmi, Gerard, Lu, Jialiang, 2021. Toward secure and efficient deep learning inference in dependable iot systems. *IEEE Int. Things J.* 8 (5), 3180–3188. <https://doi.org/10.1109/JIOT.2020.3004498>.
- Quiring, E., Rieck, K., 2020. Backdooring and poisoning neural networks with image-scaling attacks. In: *2020 IEEE Security and Privacy Workshops (SPW)*, pp. 41–47.
- Rakin, A.S., He, Z., Tbt, D. Fan, 2020. Targeted neural network attack with bit trojan. In: *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13195–13204.
- Stallkamp, Johannes, Schlipsing, Marc, Salmen, Jan, Igel, Christian, 2012. Man vs. computer: benchmarking machine learning algorithms for traffic sign recognition. *Neural Netw.* 32, 323–332. <https://doi.org/10.1016/j.neunet.2012.02.016>.
- Tang, Di, Wang, Xiaofeng, Tang, Haixu, Zhang, Kehuan, 2021. Demon in the variant: Statistical analysis of DNNs for robust backdoor contamination detection. In: *30th USENIX Security Symposium (USENIX Security 21)*, pp. 1541–1558.
- Tran, Brandon, Li, Jerry, Madry, Aleksander, 2018. Spectral signatures in backdoor attacks. In: *NeurIPS 2018. December 3–8, 2018, Montréal, Canada*, pp. 8011–8021.
- Wang, Bolun, Yao, Yuanshun, Shan, Shawn, Li, Huiying, Viswanath, B., Zheng, H., Zhao, B., 2019. Neural cleanse: identifying and mitigating backdoor attacks in neural networks. In: *2019 IEEE Symposium on Security and Privacy (SP)*, pp. 707–723.
- Wang, Ren, Zhang, Gaoyuan, Liu, Sijia, Chen, Pin-Yu, Xiong, Jinjun, Wang, Meng, 2020. Practical detection of trojan neural networks: Data-limited and data-free cases.
- Wang, Yulong, Zhao, Minghui, Li, Shenghong, Yuan, Xin, Ni, Wei, 2022a. Dispersed pixel perturbation-based imperceptible backdoor trigger for image classifier models. *IEEE Trans. Inf. Forensics Secur.* 17, 3091–3106. <https://doi.org/10.1109/TIFS.2022.3202687>.
- Wang, Zhenting, Mei, Kai, Ding, Hailun, Zhai, Juan, Ma, Shiqing, 2022b. Rethinking the Reverse-Engineering of Trojan Triggers. *Advances in Neural Information Processing Systems*, vol. 35. Curran Associates, Inc., pp. 9738–9753.
- Xu, Tong, Li, Yiming, Jiang, Yong, Xia, Shu-Tao, 2023. Batt: backdoor attack with transformation-based triggers. In: *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5.
- Xu, Xiaojun, 2020. Meta-neural-trojan-detection. <https://github.com/AI-secure/Meta-Neural-Trojan-Detection>.
- Xu, Xiaojun, Qi, Wang, Li, Huichen, Borisov, Nikita, Gunter, Carl A., Li, Bo, 2021. Detecting AI trojans using meta neural analysis. In: *42nd IEEE Symposium on Security and Privacy. SP 2021*, San Francisco, CA, USA, 24–27 May, pp. 103–120.
- Zhang, Jiale, Chen, Junjun, Wu, Di, Chen, Bing, Yu, Shui, 2019. Poisoning attack in federated learning using generative adversarial nets. In: *2019 TrustCom/BigDataSE*.
- Zhang, Jiale, Chen, Bing, Cheng, Xiang, Thi, Huynh, Binh, Thanh, Yu, Shui, 2021. Poisoning: generative poisoning attacks against federated learning in edge computing systems. *IEEE Int. Things J.* 8 (5), 3310–3322. <https://doi.org/10.1109/JIOT.2020.3023126>.
- Zhang, Kaiyuan, Tao, Guan hong, Xu, Qiuling, Cheng, Siyuan, An, Shengwei, Liu, Yingqi, Feng, Shiwei, Shen, Guangyu, Chen, Pin-Yu, Ma, Shiqing, Zhang, Xiangyu, 2023. A provable defense framework for backdoor mitigation in federated learning. In: *FLIP: The Eleventh International Conference on Learning Representations. ICLR 2023, Kigali, Rwanda, May 1–5*.
- Zhao, Shihao, Ma, Xingjun, Zheng, Xiang, Bailey, James, Chen, Jingjing, Jiang, Yu-Gang, 2020. Clean-label backdoor attacks on video recognition models. In: *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14431–14440.
- Zhao, Yi, Xu, Ke, Wang, Haiyang, Li, Bo, Jia, Ruoxi, 2021. Stability-based analysis and defense against backdoor attacks on edge computing services. *IEEE Netw.* 35 (1), 163–169. <https://doi.org/10.1109/MNET.011.2000265>.

Ying He received the B.S. degree from Nanjing University, in 2019. She is currently working toward the Ph.D. degree with the Department of Computer Science and Technology, Nanjing University. Her current research interests include security and privacy in deep learning and federated learning.

Zhili Shen received the B.S. degree in software engineering from Chongqing University, Chongqing, China, in 2020, and the M.E. degree in computer technology from Nanjing University, Nanjing, China, in 2023. His current research interests include security and privacy.

Chang Xia received the Ph.D. degree in the Department of Computer Science and Technology from Nanjing University, in 2021. His current research focuses on security and privacy in machine learning.

Jingyu Hua received the B.E. and M.E. degrees in software engineering from the Dalian University of Technology, China, in 2007 and 2009, respectively, and the Ph.D. degree in informatics from Kyushu University, Japan, in 2012. His current research interests include security and privacy in mobile computing, and system security.

Wei Tong received the B.S., M.S., and the Ph.D. degrees from Nanjing University, in 2013, 2016, and 2019, respectively. He is currently an assistant researcher with the Department of Computer Science and Technology, Nanjing University. He is interested in data privacy and information security.

Sheng Zhong received the B.S. and M.S. degrees in computer science from Nanjing University, in 1996 and 1999, respectively, and the Ph.D. degree in computer science from Yale University, in 2004. His research interests include security, privacy, and economic incentives.